



SERVICE MANAGEMENT DATABASE

An operational database, analysis and dashboard for an independent auto repair shop, rebuilt from my 2021 UNLV Information Systems capstone.

14 normalized tables (3NF)	70K rows across 24 months	22 business questions answered in SQL	20 automated data-quality tests	5 dashboard pages
--------------------------------------	-------------------------------------	---	---	-----------------------------

CONTENTS

1	Executive summary	8	Analysis & findings
2	Business problem	9	Operations dashboard
3	What changed since 2021	10	Recommendations
4	Business rules	11	Limitations & next steps
5	Database design (ERD)	A	Data dictionary
6	Normalization & design decisions	B	SQL techniques index
7	Data, simulation & quality testing	C	How to run it

Jomel Capili · B.S. Business Administration, Information Systems, University of Nevada, Las Vegas
Report date: 2026 · Data period: Jan 1, 2024 – Dec 31, 2025
Code & data: github.com/JomelCapili/Database-Design-and-Implementation
Live dashboard: jomelcapili.github.io/Database-Design-and-Implementation
Tools: MySQL 8 · SQL (CTEs, window functions) · Python · HTML/JavaScript dashboard · Power BI / Tableau build kits

All data in this report is synthetic. It was generated by a seeded Python simulation of shop operations (included in the repository) so the database could be tested at realistic scale. Every person, company, phone number and VIN is fictional.

1 EXECUTIVE SUMMARY

In 2021 I designed a service-management database for an auto repair shop as my capstone. In 2026 I audited it, found that it no longer ran and that several of its answers were wrong, and rebuilt it end to end: schema, test data, reporting layer, 22 analysis queries, data-quality tests and an interactive dashboard.

The problem

The original report named two root problems: (1) **jobs and appointments running late**, which drives customers away, and (2) **no tracking of parts usage**, which leads to stockouts. The 2021 design couldn't measure either: it linked parts to mechanics instead of jobs, and each mechanic to a single customer.

What the data shows

11%

on-time rate when a stocked part ran out, vs. 79.6% when every part was on the shelf. Those jobs took 101 hours on average instead of 6. Front brake rotors ran out 53 times in 2025 against a reorder point of 2; demand supports 10.

30.8%

of new customers came back within 6 months after a late first visit, vs. 47.9% when the first job finished on time. That gap is about 76 customers who didn't return, roughly \$29,000 in next visits at the average repair order.

7.7%

comeback (warranty rework) rate for one technician, 2.6× the rest of the team. His comebacks cost \$5,046 in unbilled labor and parts, and customers rated those original jobs 2.8 out of 5.

19.7%

no-show rate for appointments booked 15+ days ahead, vs. 3.0% for bookings made 0–2 days out.

+9.9%

net sales growth in 2025 (\$1.36M → \$1.50M), driven by a 7.6% higher average repair order rather than more cars (+1.4%). On-time completion stayed flat at about 76%, well short of a 90% target.

Top recommendations

1. **Reset reorder points** on 11 parts using lead-time demand plus safety stock. It costs about **\$3,570** in extra stock, 21% more than today's \$16.6K shelf.
2. **Build supplier ETAs into promise times** and hold the electrical supplier to its quoted lead time (it delivers late 94% of the time).
3. **Send reminders** for appointments booked 8+ days out, and cap the 8:00 AM slot at two check-ins per service writer.
4. **Add the comeback rate to technician reviews**, and keep pairing apprentices with master technicians (apprentice efficiency rose from 75% to 90%).

2 BUSINESS PROBLEM

The business is an independent, general-repair auto shop in the southwest Las Vegas valley with 6 technicians, about 4,000 repair orders a year and roughly \$1.4 million in annual sales.

The 2021 report described a shop in trouble: appointments were delayed or double-booked, mechanics couldn't work efficiently because parts were out of stock, nobody recorded which parts were used, and the resulting delays were costing customers and revenue. The shop had no database, so it couldn't study any of these patterns.

That diagnosis still holds, but the 2021 database couldn't confirm it. Answering "are late jobs costing us customers?" requires linking an appointment to a job, the job to the parts it used and the technician who did it, and the customer to their next visit. Version 2 is designed around exactly those links.

Business questions this database answers

AREA	QUESTIONS	QUERIES
Revenue & profit	How are sales and margin trending? Which services and vehicle makes drive profit? Did 2025 beat 2024?	Q1–Q4
Operations	What makes a job late? Are popular appointment slots overbooked? Who no-shows? What's on the shop floor right now?	Q5–Q8
Technicians	Who is efficient, who is accurate, is the apprentice program working, and what do comebacks cost?	Q9–Q11
Inventory	Which parts run out, what should reorder points be, do suppliers deliver on time, are parts margins holding?	Q12–Q15
Customers	Does a late job cost us the customer? How well do we retain each cohort? Which channels bring the best customers? What drives bad reviews?	Q16–Q20
Finance	How are customers paying, and who owes us money?	Q21–Q22

3 WHAT CHANGED SINCE 2021

I re-ran the original script on a standard MySQL 8.0 server on Linux. It produced **18 errors**, and several queries that did run returned wrong answers with no warning. The full audit, with the evidence for every item, is in docs/FIX_LOG.md.

2021 ISSUE	WHAT IT CAUSED	VERSION 2
Table created as Vehicle, used as VEHICLE	13 of the 18 errors on case-sensitive (Linux/macOS) MySQL	Consistent lowercase names; tested on Linux
mechanic.Customer_ID: each mechanic tied to one customer	Queries only worked because every table had exactly one row per customer	Mechanic is an employee; the work order records who did the job
Parts linked to mechanics, not jobs	Couldn't answer "which parts went into which repair", the report's own inventory problem	work_order_part links each part to its job

Revenue by make joined customer → vehicle → bill	Double counting: a test customer's \$413.72 was credited to <i>both</i> of their cars	One path: invoice → work order → vehicle
year > 2000 AND make='Honda' OR make='Toyota'	Returned 5 rows instead of 3, including 1995 and 1998 Toyotas	IN (...) and numeric model_year
"Most used part" counted rows, not units; cost ignored quantity	Wrong category ranked #1; costs understated up to 3.4×	Explicit quantity × unit_cost
No CHECK constraints; status default misspelled 'incomeplete'	Any value accepted	CHECK constraints on every code and status column
DELETes by time, price or phone number	6 of 7 DELETes failed; the one that ran removed 2 unrelated appointments	Key-based changes inside transactions; history kept

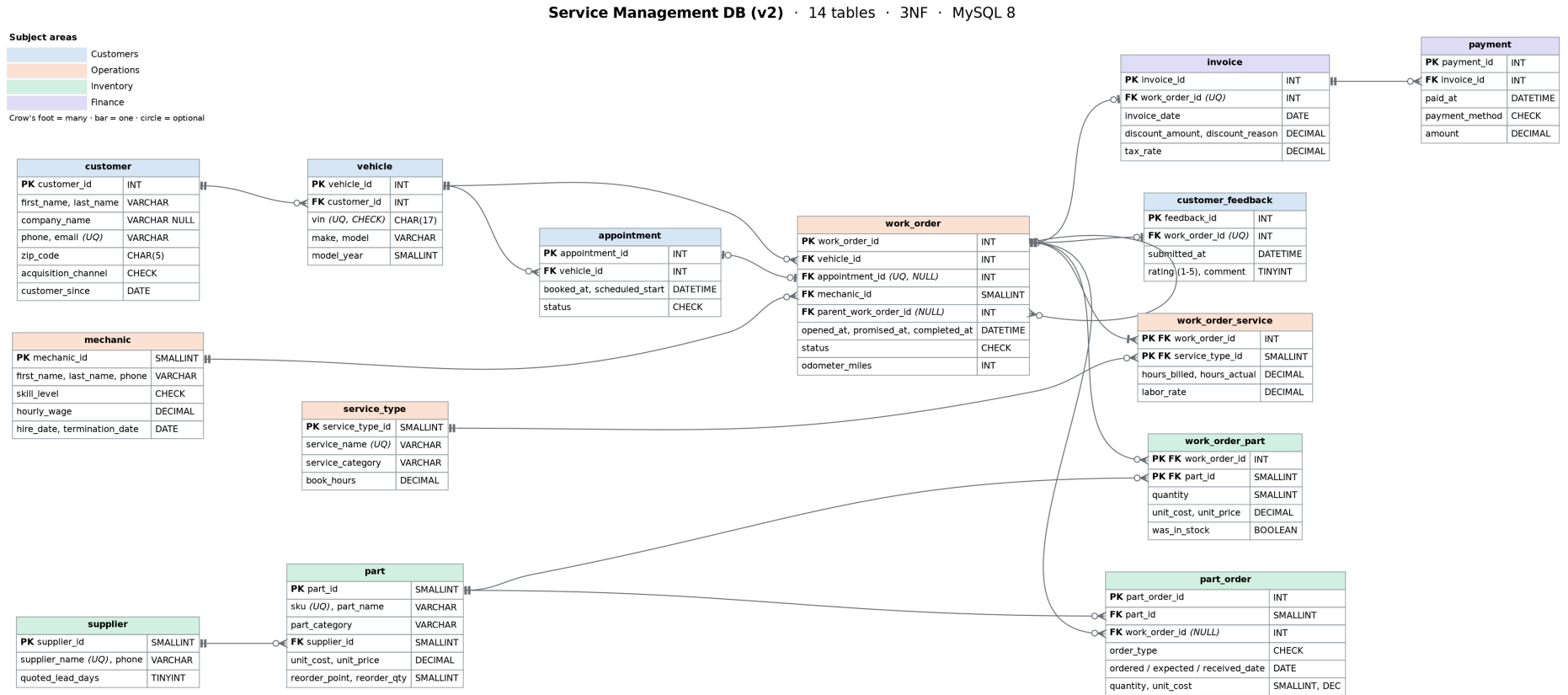
4 BUSINESS RULES

Each rule maps to a constraint in `sql/01_schema.sql` or a test in `sql/05_data_quality_checks.sql`, so the rules are enforced, not just documented.

ID	RULE	ENFORCED BY
BR-1	A customer owns one or more vehicles; each vehicle belongs to exactly one customer. A VIN is unique and valid.	FK, UNIQUE, CHECK (VIN pattern); test 1, 3
BR-2	An appointment is for one vehicle at one time. It is Scheduled, Arrived, No-Show or Cancelled.	FK, CHECK; tests 5, 6
BR-3	A work order is one visit. It may come from at most one appointment; walk-ins have none.	Nullable UNIQUE FK; test 4
BR-4	Every work order has exactly one responsible technician, who must be employed on that date. Others may help; responsibility stays with the assigned technician (2021 rule E).	NOT NULL FK; test 8
BR-5	A work order contains one or more services (labor lines) and zero or more parts, each recorded with the price at the time.	Composite PKs; price columns on lines
BR-6	A job is Completed only when it has a completion time, and the completion time is after it was opened.	CHECK constraints
BR-7	A warranty comeback points to the original job, is on the same vehicle, happens after it, and is never billed.	Self-referencing FK; tests 11, 12
BR-8	Every part that enters stock is recorded as an opening balance, a replenishment, or a special order for one job. Stock can never go negative.	CHECK; tests 18, 20
BR-9	Each completed customer-pay job has exactly one invoice, dated the day it finished. Totals are calculated, never typed.	UNIQUE FK; view; tests 10, 13
BR-10	An invoice may be paid in one or more payments but never overpaid. Retail customers pay at pickup; fleet accounts pay on terms.	FK; tests 14, 15, 16
BR-11	A customer may leave at most one rating (1–5) per job, after the job is completed.	UNIQUE FK, CHECK; test 17
BR-12	Records with history (customers, technicians, invoices) are never deleted; their state changes instead.	FKs with RESTRICT; 06_maintenance_examples.sql

5 DATABASE DESIGN

Entity relationship diagram in crow's-foot notation. Colors mark the four subject areas. Source: docs/erd.dot (Graphviz).



Entities by subject area

TABLE	WHAT ONE ROW REPRESENTS	ROWS
Customers		
customer	A retail driver or fleet account, with how they found the shop	4,052
vehicle	A car owned by a customer, identified by a validated VIN	6,360
appointment	A booking for one vehicle at one time, and whether they showed up	6,178
customer_feedback	A 1–5 star survey response for one completed job	2,781
Operations		
work_order	One visit / repair order: timing, promise, status, responsible technician	7,939
work_order_service	A labor line: one service on one job, hours billed vs. clocked	11,717
service_type	The menu of labor operations with flat-rate book hours	38
mechanic	A technician: skill level, wage, employment dates	7
Inventory		
work_order_part	A part installed on a job, its price at the time, and whether it was in stock	14,101
part	The parts catalog with reorder point and reorder quantity	49
part_order	Parts entering stock: opening balance, replenishment, or special order for a job	1,552
supplier	A parts vendor and its quoted lead time	6
Finance		
invoice	The bill for one customer-pay job; totals are derived in a view	7,649
payment	Money received against an invoice (split payments allowed)	7,719

6 NORMALIZATION & DESIGN DECISIONS

First normal form. Every column holds one value. The 2021 Customer_Name became first_name and last_name, and Appt_Date plus a text Appt_Time ('12:21 PM') became one DATETIME.

Second normal form. The two junction tables (work_order_service and work_order_part) have composite keys, and every other column depends on the whole key. Hours billed depend on *this service on this job*, not on the service alone.

Third normal form. No column depends on another non-key column:

- **No stored totals.** Invoice totals, stock on hand and gross profit are calculated in views (v_invoice_totals, v_part_inventory, v_work_order_summary). A stored total can drift from its line items; a derived one can't.
- **Customer reached through the vehicle.** appointment and work_order don't store customer_id, because the vehicle already determines the customer (a transitive dependency).

Deliberate trade-offs

- **Prices on transaction lines.** `work_order_part.unit_price` repeats the catalog price, but it's a fact about the sale: when the catalog price changes, old invoices must not change. Maintenance example 6 demonstrates this.
- **`work_order.vehicle_id` alongside `appointment_id`.** About a third of jobs are walk-ins with no appointment, so the job needs its own vehicle link. Data-quality test 4 guarantees the two never disagree.
- **Surrogate integer keys** everywhere, with natural keys (VIN, SKU, email) kept UNIQUE. The 2021 text keys such as 'C-42 ' (trailing space included) are gone.
- **Current wage for labor cost.** Labor cost uses each technician's current wage. A wage-history table would be the next step for exact historical cost.

Reporting layer

Seven views turn the normalized tables into analysis-ready facts. The dashboard, the Power BI/Tableau exports and the analysis queries all read from these views, so every tool uses one definition of every metric:

VIEW	GRAIN AND PURPOSE
<code>v_work_order_summary</code>	One row per job: revenue, cost, profit, timing, on-time flag, parts delays, comeback flag, rating. The main fact table.
<code>v_invoice_totals</code>	One row per invoice: labor, parts, discount, tax, total, paid, balance due.
<code>v_appointment_summary</code>	One row per appointment: lead time, slot, outcome, check-in wait, bookings in the same slot.
<code>v_part_inventory</code>	One row per part: on hand, on order, usage, stockouts, suggested reorder point (recursive calendar CTE for daily demand).
<code>v_mechanic_scorecard</code>	One row per technician: efficiency, on-time rate, comeback rate, rating, sales.
<code>v_monthly_kpis</code>	One row per month: the executive KPI set.
<code>v_customer_summary</code>	One row per customer: visits, lifetime sales, recency, satisfaction.

7 DATA, SIMULATION & QUALITY TESTING

The 2021 data had 50 rows per table, every relationship 1:1. That's too small to analyze and too simple to expose bugs like double counting. Version 2 is loaded with about 70,000 rows from a seeded simulation of two years of shop operations.

How the data was generated

`python/generate_data.py` (standard library only, seed 2021) simulates the shop one business day at a time:

- New and returning customers arrive. Returning customers come back roughly every four months, and are **less likely to return if their last job was late or poorly rated**.

- Customers book appointments (lead time 0–30 days) or walk in. Long-lead bookings no-show more often; the 7:30 and 8:00 AM slots fill first.
- Each visit gets 1–3 services. Seasonal demand follows Las Vegas conditions: A/C work peaks at 2–3× in summer, batteries and cooling systems about 2×.
- Each technician works one job at a time with a personal speed (efficiency) and comeback rate. Apprentices improve over time.
- Parts come off a simulated shelf with reorder points and supplier lead times. An empty shelf triggers a special order and a parts wait.
- Completed jobs are invoiced (Nevada taxes parts, not labor), paid (fleet accounts on 30-day terms) and sometimes rated.

The patterns in Section 8 were **designed into the simulation**, for example under-stocked brake rotors and one technician with a high comeback rate. The point of the project is not to discover them; it's to show that the database, queries and dashboard **can surface and quantify** problems like these from raw transactions, which is the job a data analyst does with real data.

Data-quality test suite

Constraints block most bad data at insert time. `sql/05_data_quality_checks.sql` covers the rules that span rows and tables. It runs 20 checks and returns one PASS/FAIL row per check. **All 20 pass.**

#	CHECK	TECHNIQUE
3	Every VIN's check digit (position 9) is valid	Recursive CTE over 17 positions, transliteration, weighted MOD 11
9	Odometer never goes backwards for a vehicle	LAG() over each vehicle's visits
18	Every out-of-stock part line has a matching special order, and vice versa	Correlated EXISTS
20	Stock on hand never goes negative	Running SUM() OVER across a receipts/usage ledger
1–19	Plus orphan, duplicate, date-order, invoice, payment and status-consistency checks	Anti-joins, GROUP BY ... HAVING

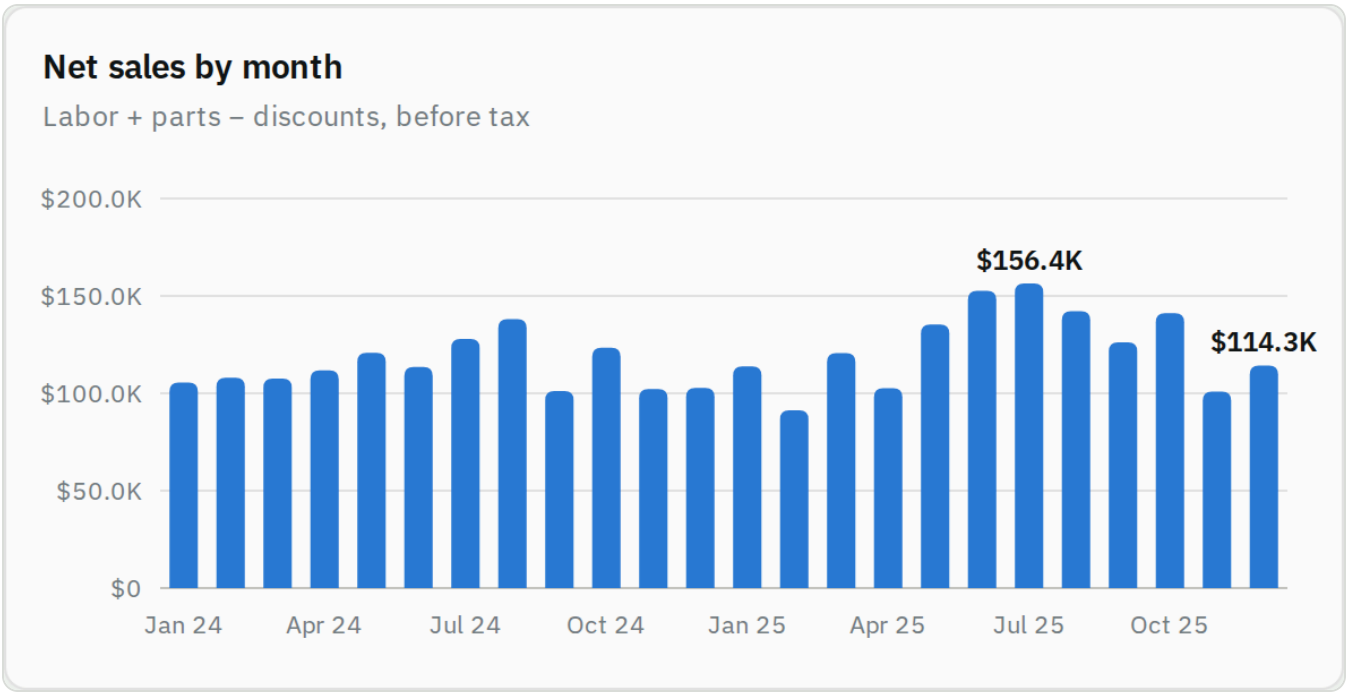
The suite caught real bugs. On its first run, 3 checks failed. One was a genuine error in my generator: a returning customer could book their next visit before their previous visit had happened. The other two were edge cases in the checks themselves: an appointment at exactly the snapshot time, and two cars finished at 11:30 AM on the snapshot day and not yet picked up. I fixed the generator, tightened the two checks, and documented each case.

8 ANALYSIS & FINDINGS

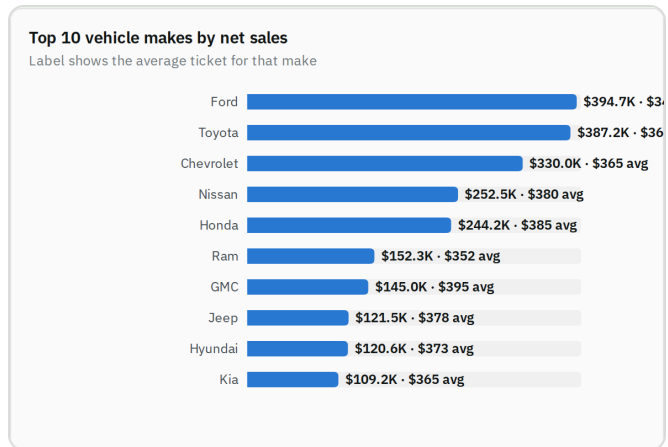
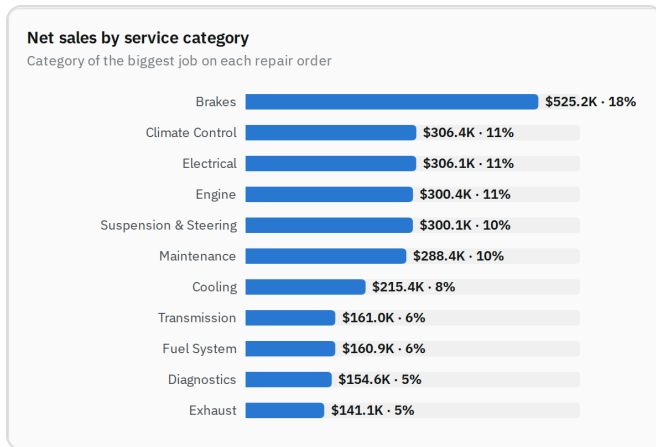
Each finding below names the query that produced it (sql/04_analysis_queries.sql). Charts come from the operations dashboard with the full 24 months selected.

8.1 Business performance: sales grew on bigger tickets, not more cars

KPI (Q2)	2024	2025	CHANGE
Car count (work orders)	3,941	3,998	+1.4%
Net sales	\$1,362,548	\$1,497,116	+9.9%
Average repair order	\$359.70	\$386.95	+7.6%
Gross margin	65.6%	66.6%	+1.0 pts
Parts margin	53.0%	52.1%	−0.8 pts
On-time completion	76.8%	76.4%	−0.3 pts
Average rating (1–5)	4.19	4.17	−0.02



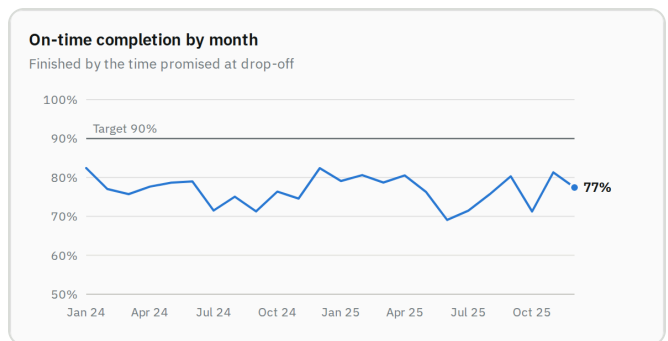
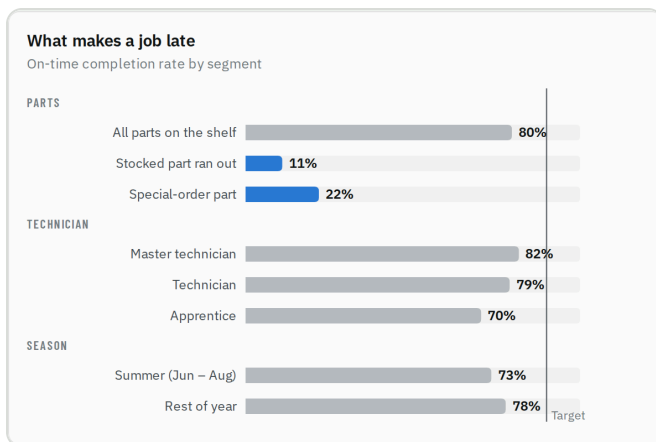
Growth came from a higher average ticket, helped by a labor-rate increase from \$140 to \$150 per hour, while car count was nearly flat. Sales peak every summer (June–August 2025 averaged \$150K a month vs. \$116K the rest of the year) because Las Vegas heat drives A/C, battery and cooling repairs (Q15: A/C work runs at 2.5× its average month in July and 0.15× in December). Brakes are the largest category at 18% of sales and the most profitable overall (Q3).



Q4 is the corrected version of 2021 query 11. European makes (BMW, Mercedes-Benz, Volkswagen, Audi) average 7–17% above the shop's typical ticket because their flat-rate labor times are longer.

8.2 Why jobs run late: parts, not people

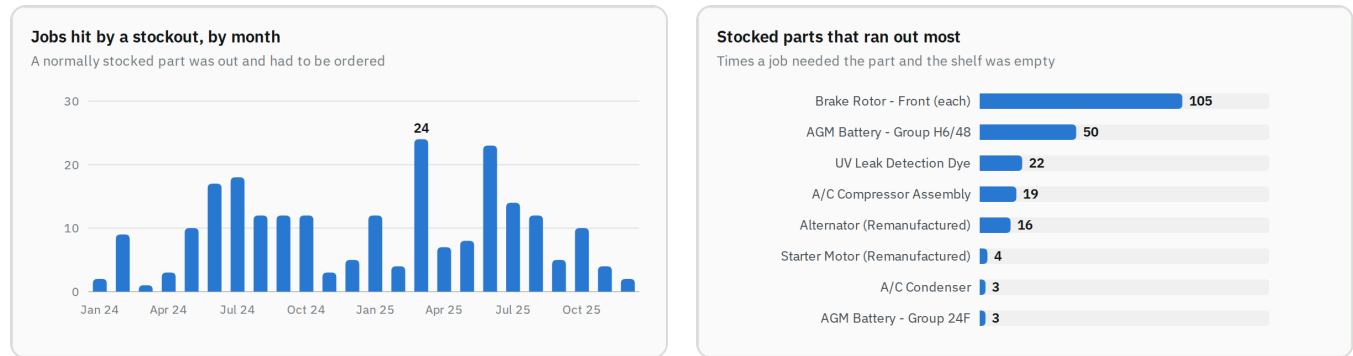
Only about 76% of jobs finish by the time promised at drop-off. Q5 splits on-time performance by every factor the database records:



Parts availability dominates. When every part was on the shelf, 79.6% of jobs finished on time. When a normally stocked part had run out, only 11.0% did, and turnaround jumped from 6 hours to 101. Technician skill matters much less (apprentices 70%, master technicians 82%). Summer's lower on-time rate (73.5%) is partly summer stockouts and partly workload: even with every part on the shelf, summer jobs were on time 77.5% of the time vs. 80.4% the rest of the year. Walk-ins and appointments perform about the same (75% vs. 77%).

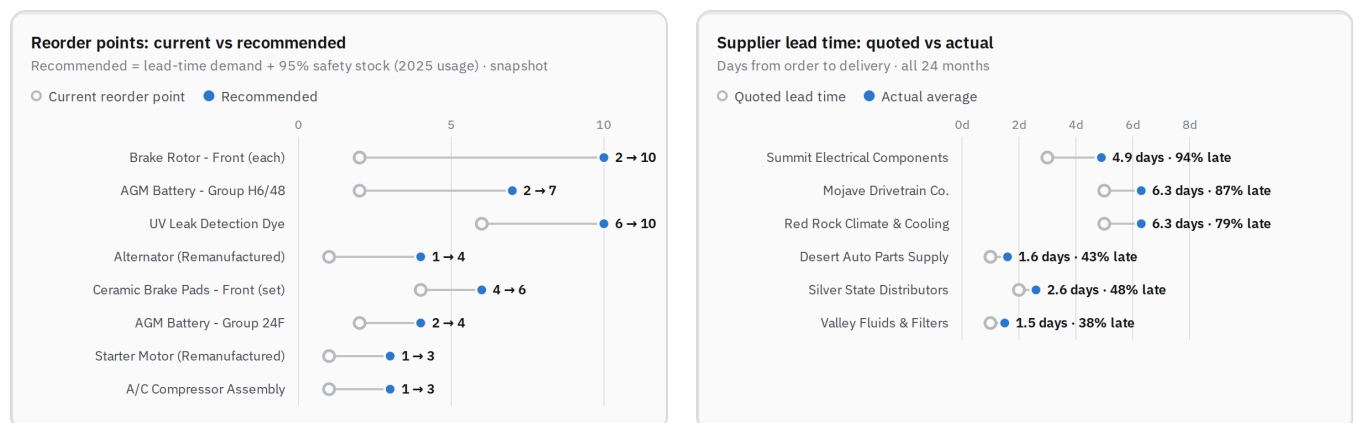
Parts delays touched only **about 5% of jobs** but account for **17% of all late jobs**. They're the most concentrated, and the most fixable, cause of the shop's core 2021 problem.

8.3 Parts stockouts: the reorder points are simply too low



Stockouts cluster in two places: **front brake rotors**, the shop's highest-volume repair part, and **summer parts** (AGM batteries, A/C compressors, leak-detection dye) whose demand doubles in the heat while reorder points stay the same. Q12 compares each part's reorder point with a recommended one calculated in `v_part_inventory`:

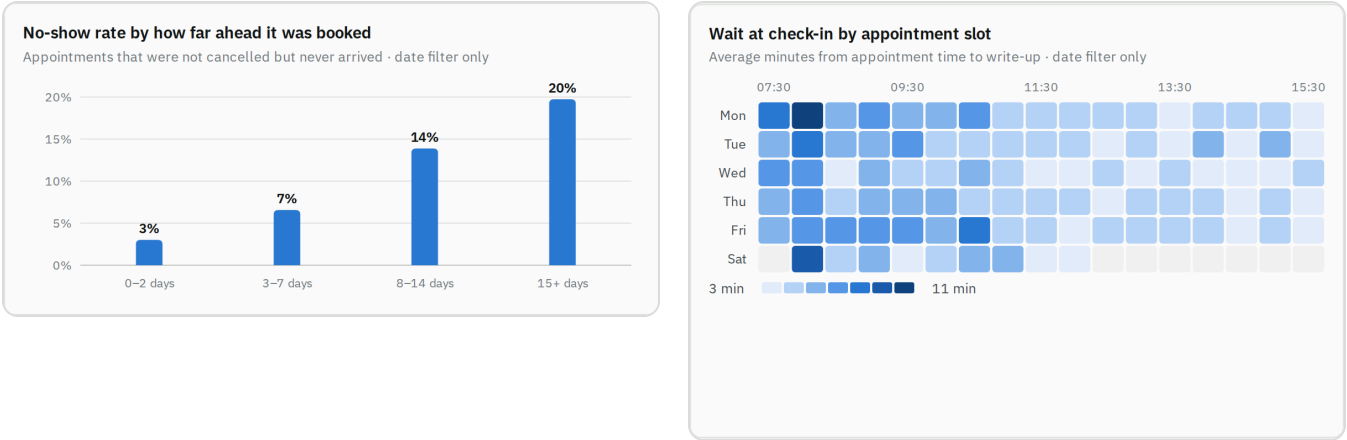
```
recommended reorder point = avg daily use × avg actual lead time
                             + 1.65 × σ(daily use) × √(lead time)      -- 95% service level
```



Front rotors need a reorder point of **10, not 2**; the Group H6 battery **7, not 2**; alternators **4, not 1**. Raising all 11 affected parts costs about **\$3,570** in extra stock, set against 229 jobs over two years that waited an average of 4.1 days for parts and rated the shop 2.5 out of 5.

Suppliers make it worse (Q13). Summit Electrical Components quotes 3 days and averages 4.9; 94% of its orders arrive late. Because the reorder-point formula uses *actual* lead times, it already compensates, but the shop should hold Summit to its quote or move volume to another supplier.

8.4 Appointments: long-lead bookings no-show, and 8 AM is overbooked

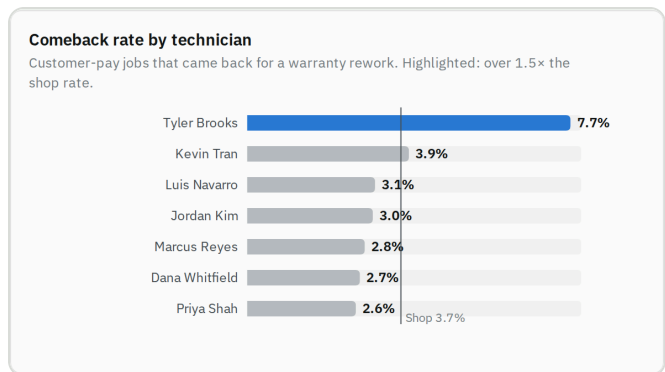
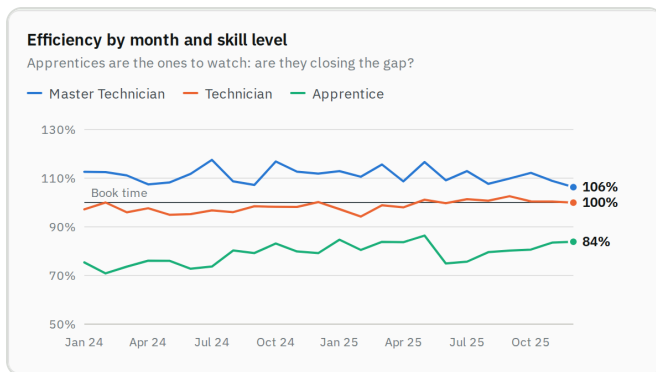


No-shows rise with booking lead time (Q7): 3.0% for appointments made 0–2 days out, 6.6% at 3–7 days, 13.9% at 8–14 days and 19.7% at 15+ days. A reminder text for anything booked a week or more ahead targets the 21% of bookings that produce nearly half (48%) of all no-shows.

The 8:00 AM slot is the 2021 "clashing appointments" problem (Q6). It averages 2.5 bookings per slot, 45% of its customers arrive into an overbooked slot, and 15% wait more than 15 minutes to be checked in, vs. under 2% after noon. With two service writers, the fix is to cap each half-hour slot at two check-ins and move the overflow to 8:30–9:30.

8.5 Technicians: one quality outlier, and a working apprentice program

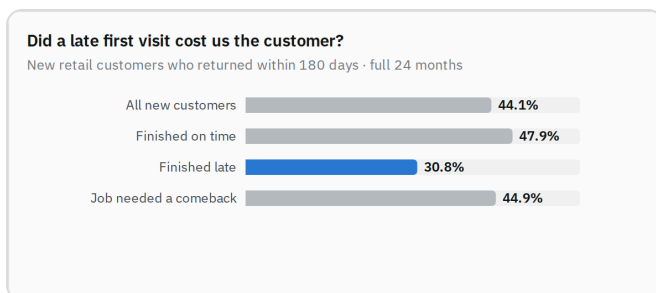
Technician scorecard							
Efficiency = flat-rate hours billed ÷ hours clocked. The tick marks 100% (book time).							
TECHNICIAN	JOBS	EFFICIENCY	ON-TIME	COMEBACK RATE	RATING	NET SALES	ACTION
Marcus Reyes Master Technician	771	113.9%	83.1%	2.82%	4.23	\$421.2K	✓ On track
Dana Whitfield Master Technician	696	108.4%	80.2%	2.70%	4.33	\$375.7K	✓ On track
Priya Shah Technician	1,458	102.2%	81.1%	2.60%	4.24	\$551.7K	✓ On track
Luis Navarro Technician	1,386	99.4%	77.9%	3.06%	4.19	\$503.0K	✓ On track
Tyler Brooks Technician	958	90.8%	77.1%	7.74%	4.10	\$328.1K	! Quality review
Kevin Tran Apprentice	2,081	81.0%	71.8%	3.87%	4.13	\$523.4K	🔧 Coaching
Jordan Kim Apprentice	589	72.7%	65.5%	3.01%	4.05	\$156.5K	🔧 Coaching



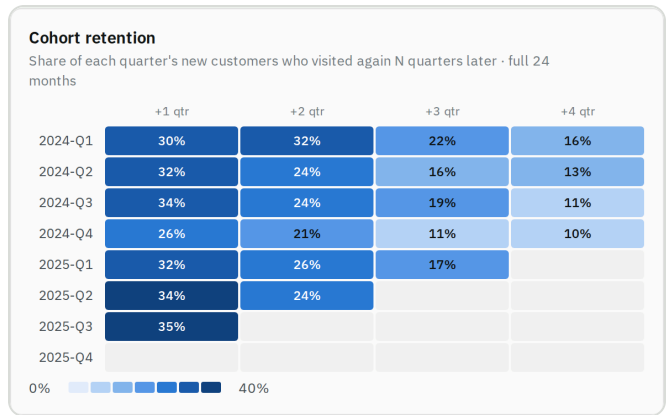
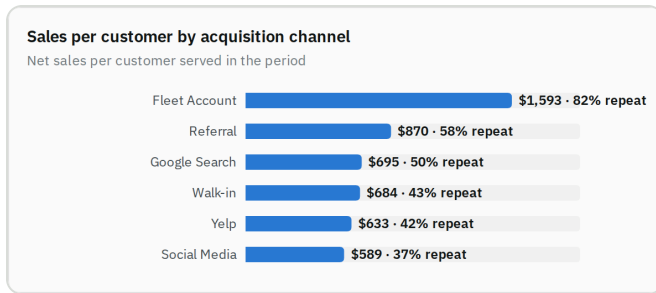
Efficiency (flat-rate hours billed ÷ hours clocked) ranges from 114% for the senior master technician to 73% for the newest apprentice (Q9). **Quality is a separate question.** Tyler Brooks was an average-speed technician (91%) but had a **7.7% comeback rate**, 2.6× everyone else. His 70 comebacks cost \$5,046 in unbilled rework (Q11, a self-join from each rework to its original job), and customers rated those original jobs 2.8 out of 5. He left in May 2025.

The apprentice program works (Q10). Kevin Tran's efficiency rose every quarter, from 74.6% in Q1 2024 to 89.5% in Q4 2025 (a LAG() window over quarterly results). Jordan Kim, hired in June 2025, is on the same curve (69% → 79% in two quarters).

8.6 Customers: a late first visit costs a third of repeat business



This is the central finding (Q16). Among new retail customers who had six months to come back, 47.9% returned after an on-time first visit but only **30.8% after a late one**. It uses ROW_NUMBER() to find each customer's first visit and LEAD() to find their next one. Applied to the 445 customers whose first job was late, the gap is about 76 lost returning customers, or roughly \$29,000 in next visits at the 2025 average repair order of \$387, before counting lifetime value. Ratings tell the same story (Q20): 4.47 when on time, 3.21 when late, and 2.75 after waiting on an ordered part, with 43% of those reviews at 1–2 stars.

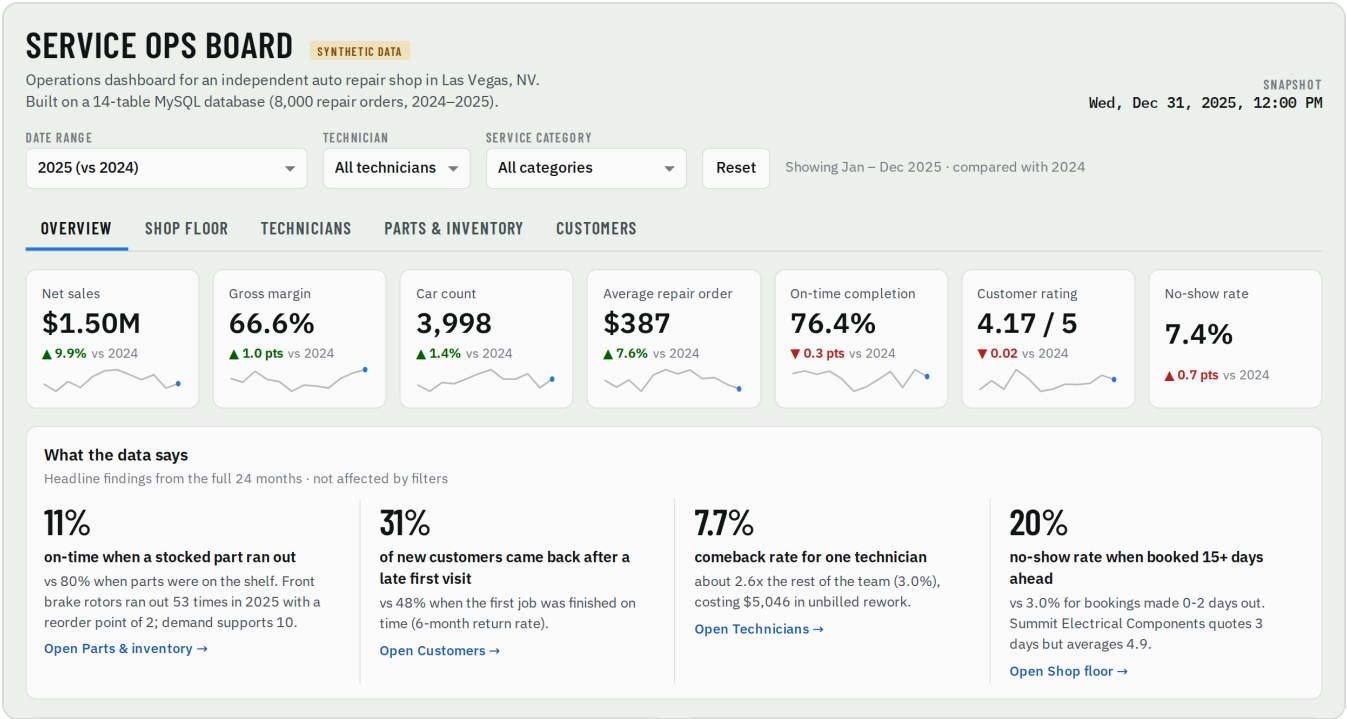


Referrals bring the best retail customers (Q18): a 56.6% repeat rate and \$798 in sales per customer, vs. 38.1% and \$573 from social media. Fleet accounts are the most valuable of all (\$1,477 per account, 81% repeat). The cohort view (Q17) shows about a third of each quarter's new customers returning the following quarter, and about 10–16% still visiting a year later. RFM segmentation (Q19, NTILE(5)) finds 17% of customers ("Champions") produce 36% of sales, and 430 "at risk" high-value customers average 17 months since their last visit, a ready-made win-back list.

Finance (Q21–Q22). Mobile-wallet payments grew from 7.6% to 12.8% of transactions while cash fell from 11.7% to 8.4%. Accounts receivable is small and current: \$13,260 owed by fleet accounts, 85% of it under 30 days old.

9 OPERATIONS DASHBOARD

The dashboard turns the reporting views into a tool a shop manager could use every morning. It is a single self-contained web page (docs/index.html), published free with GitHub Pages, with no server or login.



10 RECOMMENDATIONS

Ranked by expected impact on the two original problems. Each has a metric on the dashboard so the shop can tell whether it worked.

#	ACTION	EVIDENCE	TRACK
1	Raise reorder points on 11 parts to the recommended levels; use higher summer reorder points (June–August) for batteries and A/C parts.	Stockout jobs: 11% on-time, 101-hour turnaround, 2.5★. Cost: ~\$3,570 in added stock.	Jobs hit by a stockout; reorder watchlist
2	Promise times that include parts ETAs , plus a text update when a job will be late.	Late first visit: 30.8% return vs. 47.9% (≈76 lost customers, ≈\$29K).	On-time %; 6-month return rate
3	Supplier accountability: hold Summit Electrical to its 3-day quote or shift volume.	94% of Summit orders late; average 4.9 days.	Supplier late %; actual lead days
4	Appointment reminders for bookings 8+ days out; cap 8:00 AM at two check-ins per writer.	No-shows 13.9–19.7% vs. 3.0%; 15% of 8 AM customers wait 15+ min.	No-show rate; check-in heatmap
5	Add comeback rate to technician reviews; flag anyone over 1.5× the shop rate for a quality review.	One technician at 7.7% (2.6× peers) cost \$5,046 in rework.	Comeback rate by technician
6	Keep pairing apprentices with master technicians and give apprentices longer promise buffers.	Apprentice efficiency 75% → 90% in 8 quarters; apprentice on-time 70%.	Efficiency by skill level
7	Fund a referral program and a win-back campaign for "at risk" high-value customers.	Referral customers: 57% repeat, \$798 each; 430 lapsed high-value customers.	Sales per customer by channel; RFM segments
8	Re-price electrical parts to protect margin.	Electrical parts margin fell 2.8 pts after a 9% supplier cost increase (Q14).	Parts margin by category

11 LIMITATIONS & NEXT STEPS

- **Synthetic data.** The patterns are realistic but were designed into the simulation. The same schema, queries and dashboard would run unchanged on a real shop's data.
- **Simplifications.** One location, current wages for labor cost, and no per-line time tracking (clocked hours are recorded per service, but technicians aren't split within a job).
- **Next steps.** Add wage history and a promise-time audit trail; schedule a nightly refresh from the shop system (Python + cron, or dbt on a cloud warehouse); forecast weekly parts demand to set seasonal reorder points automatically; build a churn model that predicts which customers won't return.

A DATA DICTIONARY

Generated from MySQL's `information_schema`, so it always matches the deployed schema. PK = primary key, FK = foreign key, UQ = unique.

customer

COLUMN	TYPE	NULL	KEY	NOTES
customer_id	INT UNSIGNED		PK, auto	
first_name	VARCHAR(50)			
last_name	VARCHAR(50)			
company_name	VARCHAR(80)	yes		Fleet accounts only (CHECK ties it to the channel)
phone	VARCHAR(20)	yes		
email	VARCHAR(120)	yes	UQ	
zip_code	CHAR(5)			
acquisition_channel	VARCHAR(20)			How the customer found the shop
customer_since	DATE			First contact (booking or visit)

CHECK constraints: `ck_customer_channel`, `ck_customer_fleet`, `ck_customer_zip`

vehicle

COLUMN	TYPE	NULL	KEY	NOTES
vehicle_id	INT UNSIGNED		PK, auto	
customer_id	INT UNSIGNED		FK → customer.customer_id	
vin	CHAR(17)		UQ	17 chars, no I/O/Q; check digit verified by DQ test 3
make	VARCHAR(30)			
model	VARCHAR(40)			
model_year	SMALLINT UNSIGNED			

CHECK constraints: `ck_vehicle_vin`, `ck_vehicle_year`

appointment

COLUMN	TYPE	NULL	KEY	NOTES
appointment_id	INT UNSIGNED		PK, auto	
vehicle_id	INT UNSIGNED		FK → vehicle.vehicle_id	
booked_at	DATETIME			When the booking was made
scheduled_start	DATETIME			
status	VARCHAR(10)			Scheduled, Arrived, No-Show, Cancelled

CHECK constraints: ck_appointment_order, ck_appointment_status

customer_feedback

COLUMN	TYPE	NULL	KEY	NOTES
feedback_id	INT UNSIGNED		PK, auto	
work_order_id	INT UNSIGNED		UQ, FK → work_order.work_order_id	
submitted_at	DATETIME			
rating	TINYINT UNSIGNED			1–5 stars
comment	VARCHAR(255)	yes		

CHECK constraints: ck_feedback_rating

work_order

COLUMN	TYPE	NULL	KEY	NOTES
work_order_id	INT UNSIGNED		PK, auto	
vehicle_id	INT UNSIGNED		FK → vehicle.vehicle_id	
appointment_id	INT UNSIGNED	yes	UQ, FK → appointment.appointment_id	NULL for walk-ins
mechanic_id	SMALLINT UNSIGNED		FK → mechanic.mechanic_id	The single responsible technician
parent_work_order_id	INT UNSIGNED	yes	FK → work_order.work_order_id	Set on warranty reworks: the original job
opened_at	DATETIME			
promised_at	DATETIME			Completion time promised at drop-off
completed_at	DATETIME	yes		
status	VARCHAR(20)			Open, In Progress, Waiting on Parts, Completed
odometer_miles	INT UNSIGNED			

CHECK constraints: ck_work_order_complete, ck_work_order_promise, ck_work_order_status,
ck_work_order_status_date

work_order_service

COLUMN	TYPE	NULL	KEY	NOTES
work_order_id	INT UNSIGNED		PK, FK → work_order.work_order_id	
service_type_id	SMALLINT UNSIGNED		PK, FK → service_type.service_type_id	
hours_billed	DECIMAL(5,2)			Flat-rate hours charged (0 on warranty rework)
hours_actual	DECIMAL(5,2)			Hours the technician clocked

labor_rate	DECIMAL(6,2)	Shop rate at the time (\$140 in 2024, \$150 in 2025)
------------	--------------	--

CHECK constraints: ck_wos_hours, ck_wos_rate

service_type

COLUMN	TYPE	NULL	KEY	NOTES
service_type_id	SMALLINT UNSIGNED		PK	
service_name	VARCHAR(60)		UQ	
service_category	VARCHAR(30)			
book_hours	DECIMAL(4,2)			Standard flat-rate labor time

CHECK constraints: ck_service_type_hrs

mechanic

COLUMN	TYPE	NULL	KEY	NOTES
mechanic_id	SMALLINT UNSIGNED		PK, auto	
first_name	VARCHAR(50)			
last_name	VARCHAR(50)			
phone	VARCHAR(20)			
skill_level	VARCHAR(20)			
hourly_wage	DECIMAL(6,2)			Current wage; used for labor cost
hire_date	DATE			
termination_date	DATE	yes		

CHECK constraints: ck_mechanic_dates, ck_mechanic_skill, ck_mechanic_wage

work_order_part

COLUMN	TYPE	NULL	KEY	NOTES
work_order_id	INT UNSIGNED		PK, FK → work_order.work_order_id	
part_id	SMALLINT UNSIGNED		PK, FK → part.part_id	
quantity	SMALLINT UNSIGNED			
unit_cost	DECIMAL(8,2)			Cost at the time of use
unit_price	DECIMAL(8,2)			Price charged (0 on warranty rework)
was_in_stock	TINYINT(1)			FALSE = special-ordered for this job

CHECK constraints: ck_wop_amounts, ck_wop_quantity

part

COLUMN	TYPE	NULL	KEY	NOTES
part_id	SMALLINT UNSIGNED		PK	
sku	VARCHAR(20)		UQ	
part_name	VARCHAR(80)			
part_category	VARCHAR(30)			
supplier_id	SMALLINT UNSIGNED		FK → supplier.supplier_id	
unit_cost	DECIMAL(8,2)			
unit_price	DECIMAL(8,2)			
reorder_point	SMALLINT UNSIGNED			Reorder when on hand + on order falls to this
reorder_qty	SMALLINT UNSIGNED			0 = special-order only, never stocked

CHECK constraints: ck_part_cost, ck_part_markup

part_order

COLUMN	TYPE	NULL	KEY	NOTES
part_order_id	INT UNSIGNED		PK, auto	
part_id	SMALLINT UNSIGNED		FK → part.part_id	
work_order_id	INT UNSIGNED	yes	FK → work_order.work_order_id	Set only for special orders
order_type	VARCHAR(15)			
ordered_date	DATE			
expected_date	DATE			
received_date	DATE	yes		NULL = still in transit
quantity	SMALLINT UNSIGNED			
unit_cost	DECIMAL(8,2)			

CHECK constraints: ck_part_order_dates, ck_part_order_qty, ck_part_order_special, ck_part_order_type

supplier

COLUMN	TYPE	NULL	KEY	NOTES
supplier_id	SMALLINT UNSIGNED		PK	
supplier_name	VARCHAR(80)		UQ	
phone	VARCHAR(20)			
quoted_lead_days	TINYINT UNSIGNED			Lead time the supplier promises

invoice

COLUMN	TYPE	NULL	KEY	NOTES
invoice_id	INT UNSIGNED		PK, auto	
work_order_id	INT UNSIGNED		UQ, FK → work_order.work_order_id	
invoice_date	DATE			
discount_amount	DECIMAL(8,2)			
discount_reason	VARCHAR(40)	yes		Required whenever a discount is given
tax_rate	DECIMAL(6,5)			Applied to parts only (Nevada does not tax repair labor)

CHECK constraints: ck_invoice_discount, ck_invoice_tax

payment

COLUMN	TYPE	NULL	KEY	NOTES
payment_id	INT UNSIGNED		PK, auto	
invoice_id	INT UNSIGNED		FK → invoice.invoice_id	
paid_at	DATETIME			
payment_method	VARCHAR(20)			Card, cash, mobile wallet, or ACH / check (fleet)
amount	DECIMAL(9,2)			

CHECK constraints: ck_payment_amount, ck_payment_method

B SQL TECHNIQUES INDEX

TECHNIQUE	WHERE
Common table expressions (CTEs)	All 7 views; Q1–Q3, Q10, Q12, Q14–Q17, Q19, Q21; DQ suite
Recursive CTE	v_part_inventory (business-day calendar for demand variability); DQ check 3 (VIN positions)
Window functions: LAG, LEAD	Q1 (month-over-month and year-over-year), Q10 (quarter-over-quarter), Q16 (next visit), DQ check 9
Window functions: ROW_NUMBER, RANK, NTILE	v_work_order_summary (primary service), Q3, Q9, Q16, Q19 (RFM quintiles)
Window aggregates: SUM/AVG/COUNT ... OVER	Q1 (YTD), Q3–Q4 (share of total), Q9 (vs. shop average), Q15, Q19, v_appointment_summary, DQ check 20 (running balance)
Conditional aggregation (pivots)	Q2, Q7, Q14, Q15 (12-month pivot), Q17 (cohort matrix), Q21
Self-join	Q11 (rework → original job); DQ check 12
Correlated subqueries, EXISTS / NOT EXISTS	Q8; DQ checks 1, 11, 18, 19; maintenance examples 10–11
Statistics in SQL	STDDEV_SAMP and SQRT for safety stock in v_part_inventory
Transactions	06_maintenance_examples.sql: 11 safe UPDATE/DELETE patterns, each ending in ROLLBACK
Constraints	14 PKs, 16 FKs (including a self-reference), 8 UNIQUEs, 30 CHECKs (some with REGEXP_LIKE)

C HOW TO RUN IT

```
# 1. Build the database (MySQL 8.0+). Runs in about 10 seconds.
mysql -u root -p < sql/01_schema.sql
mysql -u root -p < sql/02_seed_data.sql
mysql -u root -p < sql/03_views.sql

# 2. Ask questions and test the data
mysql -u root -p -t < sql/04_analysis_queries.sql
mysql -u root -p -t < sql/05_data_quality_checks.sql      # expect 20 x PASS
mysql -u root -p -t < sql/06_maintenance_examples.sql    # rolls itself back

# 3. Optional: regenerate data, export for BI tools, rebuild the dashboard
python3 python/generate_data.py                        # standard library only
pip install pymysql openpyxl
python3 python/export_bi_data.py                        # data/bi/*.csv + .xlsx
python3 python/build_dashboard.py                       # docs/index.html
```

MySQL Workbench users: open each file with *File* → *Open SQL Script* and run it with the lightning-bolt button, in the order above.